

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**МНОГООБЛАЧНО РАЗГРЪЩАНЕ НА
ИЗЧИСЛИТЕЛНО ИНТЕНЗИВНА УСЛУГА ОТ
ЕДНО ДЕКЛАРАТИВНО ОПИСАНИЕ НА
ИНФРАСТРУКТУРАТА И СРАВНИТЕЛНА
ОЦЕНКА НА ДВА ОБЛАЧНИ ДОСТАВЧИКА**

КУРСОВ ПРОЕКТ

по дисциплина „Облачни изчисления и технологии“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Доц. д-р Антония Ташева

София, 2027

СЪДЪРЖАНИЕ

УВОД	1
I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД	3
1.1. Сигурност при работа с облачни технологии	4
1.2. Изводи от прегледа	5
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	6
2.1. Начин на изпълнение на изчислителния работник	6
2.2. Инструмент за декларативно описание на инфраструктурата	6
2.3. Генератор на синтетично натоварване	7
2.4. Събиране на метрики	7
2.5. Обобщение на направените избори	7
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	9
3.1. Модел на идентичността и достъпа	9
3.2. Функционална схема	10
3.3. Договор на модулите за инфраструктура	10
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	11
4.1. Контейнеризация	11
4.2. Описание на инфраструктурата	12
4.3. Мерки срещу изтичане на данни за достъп	12
V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНО СРАВНЕНИЕ	13
5.1. Условия за валидност на едно измерване	13
5.2. Планирани експерименти	13
5.3. Отчитани величини	14
5.4. Известни ограничения на методиката	14
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	15
6.1. Закъснение при постоянно натоварване	15
6.2. Поведение при хоризонтално мащабиране	15
6.3. Цена за единица извършена работа	15

6.4. Обобщен анализ	16
ЗАКЛЮЧЕНИЕ	17
ЛИТЕРАТУРА	18
ПРИЛОЖЕНИЯ	20

УВОД

Публичните облачни платформи се описват в маркетинговите си материали като взаимозаменяеми: една и съща услуга може да бъде разгърната при кой да е доставчик, а изборът между тях се свежда до цена. На практика взаимозаменяемостта е ограничена. Всеки доставчик предлага собствен набор от управлявани услуги, собствен модел на идентичността, собствени единици за таксуване и собствено поведение при автоматично мащабиране. Твърдението, че две среди са еквивалентни, обикновено се приема без проверка, защото проверката изисква едно и също натоварване да бъде разгърнато и измерено на две места при съпоставими условия.

Настоящият курсов проект изгражда *Stratus*, изчислително интензивна услуга, опакована в контейнер и разгръщана едновременно в Amazon Web Services и Microsoft Azure от едно декларативно описание на инфраструктурата. Услугата приема заявки за изчисление, чете и записва входните и изходните си данни в обектно съхранение и се мащабира хоризонтално под синтетично натоварване. Двете разгръщания се различават единствено по стойността на един параметър, което прави сравнението между тях осмислено: измерва се различието между доставчиците, а не различието между две ръчно настроени системи.

Целта на проекта е да провери до каква степен едно декларативно описание на инфраструктурата е достатъчно за разгръщането на една и съща услуга при два независими доставчика, и да измери разликата между тях по закъснение, поведение при мащабиране и цена за единица извършена работа.

За постигането на целта са формулирани следните **задачи**:

1. преглед на фундаменталните понятия в облачните изчисления, на моделите на обслужване IaaS, PaaS и SaaS, на видовете виртуализация и на клъстерните и GRID технологиите, върху които стъпват съвременните платформи;
2. анализ на възможните решения за изчислителна услуга, за инструмент за декларативно описание на инфраструктурата и за генератор на натоварване, и обосновка на направения избор;
3. проектиране на архитектура, в която платформено зависимата част е сведена до отделни модули, а описанието на услугата остава общо за двата доставчика;
4. програмна реализация на изчислителния работник, на контейнера и на модулите за

инфраструктура, заедно със събирането на метрики;

5. изграждане на методика за сравнително измерване при съпоставими условия;
6. провеждане на измерванията и анализ на получените резултати.

Обхватът на проекта е ограничен до един тип изчислително натоварване, до два доставчика и до два региона в Европа. Извън обхвата остават хибридните и частните облаци, миграцията на състояние между доставчиците по време на работа, както и услугите за управление на бази от данни. Всички идентификатори на сметки, абонаменти и ключове за достъп остават извън хранилището с изходния текст и се подават чрез променливи на средата.

Структура на изложението. Раздел I разглежда предметната област и литературните източници. Раздел II излага разгледаните алтернативи и обосновава избора между тях. Раздел III представя проектирането на системата, а Раздел IV нейната програмна реализация. Раздел V описва методиката за измерване, а Раздел VI получените резултати и техния анализ.

I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

Понятието *облачни изчисления* има работно определение, върху което стъпва по-голямата част от литературата и на което се позовават самите доставчици. То описва модел за достъп при поискване до споделен резерв от изчислителни ресурси, които се предоставят и връщат обратно бързо и с минимално административно взаимодействие, и извежда пет съществени характеристики: самообслужване при поискване, широк мрежов достъп, обединяване на ресурсите, бърза еластичност и измерена услуга [1]. Последните две са пряко относими към този проект: еластичността е това, което се проверява при синтетичното натоварване, а измерената услуга е това, което прави сравнението по цена възможно изобщо.

Същият източник разграничава трите модела на обслужване. При *инфраструктура като услуга* (IaaS) потребителят получава изчислителни възли, мрежа и съхранение и сам отговаря за операционната система и всичко над нея. При *платформа като услуга* (PaaS) доставчикът поема средата за изпълнение, а потребителят предоставя само приложението. При *софтуер като услуга* (SaaS) потребителят използва готово приложение и не управлява нищо от инфраструктурата под него. Границата между IaaS и PaaS в съвременните управлявани услуги за контейнери е размита и точно тази размита граница е една от причините сравнението между двамата доставчици да не е тривиално: еднакво наименована услуга при двамата може да лежи от различни страни на границата. Ранният анализ на икономиката на облака [2] формулира и защо еластичността има стойност, надхвърляща средната утилизация: тя премахва нуждата от оразмеряване по върховото натоварване.

Технологичната основа на обединяването на ресурси е *виртуализацията*. Класическата хардуерна виртуализация чрез хипервайзор изолира пълни виртуални машини, всяка със собствено ядро, и е описана подробно за хипервайзора Xen [3]. Виртуализацията на ниво операционна система, известна като контейнеризация, споделя едно ядро между изолирани пространства от имена и постига значително по-кратко време за стартиране за сметка на по-слаба изолация [4]. Между двата подхода стоят леките виртуални машини, проектирани специално за многонаемни среди с кратко живеещи задачи [5]. Изборът между тези три степени на изолация определя както времето за

реакция при мащабиране, така и профила на риска при сигурността, и затова се разглежда в Раздел II.

Управлението на голям брой еднакви изчислителни възли води до *клъстерните технологии*. Индустриалният опит с планировчик за клъстери в мащаба на един голям доставчик е описан в [6], а изведените от него уроци и връзката им със следващото поколение системи за оркестрация са обобщени в [7]. Историческият предшественик на този клас системи са *GRID технологиите*, чиято цел е обединяването на ресурси между отделни административни организации и които въвеждат понятието виртуална организация [8]. Разликата между GRID и облака е преди всичко в собствеността и модела на таксуване, а не в техническата задача, и това е полезно за настоящия проект, защото показва, че управлението на разпределени ресурси е по-стара задача от търговските облачни платформи.

Съхранението в облака е отделен клас проблеми. Обектните хранилища жертват част от семантиката на файловата система в замяна на висока достъпност и практически неограничен обем. Два основополагащи текста описват вътрешното устройство на такива системи при два различни доставчика и правят различен избор по оста достъпност спрямо съгласуваност [9, 10]. За настоящия проект този избор има пряко следствие: ако входните и изходните данни на изчислителния работник се четат и пишат в обектно хранилище, поведението при едновременен достъп зависи от гаранциите на конкретния доставчик, а не от кода на работника.

1.1. Сигурност при работа с облачни технологии

Сигурността в облака се управлява по модела на споделената отговорност: доставчикът отговаря за сигурността на самата платформа, а потребителят за всичко, което разгръща върху нея. Практическото следствие за този проект е отказът от дълготрайни ключове за достъп в полза на *управлявана идентичност*, при която изчислителният възел получава краткосрочен маркер за достъп от самата платформа и никакъв секрет не се записва в хранилището с изходния текст. Точните имена на механизмите при двамата доставчици и разликите между тях се описват в Раздел III.

[TODO: Да се допълни: сравнителна таблица на моделите на идентичност при двамата доставчици, с препратка към официалната документация на всеки от тях.]

1.2. Изводи от прегледа

Прегледът показва, че фундаменталните понятия са общи за двете разглеждани платформи, докато конкретните механизми се различават. Това оправдава подхода на проекта: общото се изразява веднъж, в декларативно описание, а различното се изнася в отделни модули. Прегледът показва също, че липсва достъпно сравнение на двата доставчика при контролирано еднакво натоварване, което е приносът, който този проект се опитва да даде в рамките на своя ограничен обхват.

[TODO: Да се посочат конкретните публикувани сравнения между доставчици, ако при допълнителното търсене бъдат намерени такива, и да се уточни в какво настоящата постановка се различава от тях.]

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

Изборите в този проект са четири: как се изпълнява изчислителният работник, как се описва инфраструктурата, как се създава натоварването и как се събират метриките. Всеки от тях се оценява по три критерия, общи за целия проект: наличие на съпоставим еквивалент при двамата доставчици, възможност описанието да остане едно, и пригодност на получените измервания за сравнение.

2.1. Начин на изпълнение на изчислителния работник

Разгледани са три възможности. Първата е *виртуални машини* с ръчно инсталирано приложение, тоест чист IaaS. Тя дава пълен контрол и почти еднакво поведение при двамата доставчици, но времето за стартиране на нов възел е от порядъка на минути, което прави наблюдението на еластичността бавно и скъпо. Втората е *управлявана услуга за контейнери* без достъп до възлите под нея. Тя намалява административната работа, но всеки доставчик я оформя различно и еднаквото описание става трудно. Третата е *Kubernetes* в управляван вид, тоест Amazon EKS и Azure AKS, при която описанието на самото приложение е идентично и различно остава само описанието на клъстера.

Избран е третият вариант. Той е единственият, при който описанието на работното натоварване буквално съвпада при двамата доставчици, което е предпоставка за осмислено сравнение. Цената на избора е по-висока постоянна такса за управляващия слой и по-дълго първоначално разгръщане, и двете се отчитат при анализа на разходите.

2.2. Инструмент за декларативно описание на инфраструктурата

Скриптовете на командните интерфейси на самите доставчици отпадат веднага: те са императивни, не са идемпотентни и не позволяват общо описание. Собствените средства на доставчиците, AWS CloudFormation и Azure Bicep, са зрели, но всеки от тях работи само при своя доставчик, което би означавало две несвързани описания. Остават инструментите с множество доставчици: Terraform и неговото разклонение OpenTofu, и Pulumi, при който инфраструктурата се описва на език за общо предназначение.

Избран е Terraform, съответно OpenTofu. Причината е, че при него разликата между двата доставчика се локализира в отделни модули, а общата част, тоест

променливите, изходите и редът на разгръщане, остава една. Pulumi предлага същото, но въвежда зависимост от среда за изпълнение на език, който иначе не участва в проекта. Разликата между Terraform и OpenTofu е лицензионна, не техническа, и изборът между тях не влияе на резултатите.

2.3. Генератор на синтетично натоварване

Изискването към генератора е едно: да произвежда еднакво натоварване срещу двете разгръщания и да отчита закъснението по персентили, а не само средна стойност. Разгледани са wrk, k6 и Apache JMeter. Първият е най-лек и дава най-малко собствено изкривяване, но описанието на сценария при него е ограничено. Третият е най-богат, но е тежък и изисква отделна инфраструктура за самия себе си.

Избран е k6, защото сценарият се описва в текстов файл, който се версионира заедно с останалата част на проекта, и защото инструментът извежда персентили директно. **[TODO: Да се потвърди, че генераторът се пуска от една и съща машина срещу двете разгръщания, и да се опише как се отчита разликата в мрежовото разстояние до двата региона.]**

2.4. Събиране на метрики

Собствените средства за наблюдение на доставчиците, Amazon CloudWatch и Azure Monitor, са налични без допълнителна работа, но измерват различни величини по различен начин и затова не са годни за пряко сравнение. Затова метриките се събират от самото приложение в общ формат, а средствата на доставчика се използват само за контролна проверка.

Избрани са Prometheus и Grafana, разгърнати в самия клъстер. Така двете разгръщания се наблюдават с един и същ код и една и съща конфигурация, а единствената разлика остава средата под тях.

2.5. Обобщение на направените избори

[TODO: Да се добави обобщаваща таблица със структура: решение, разгледани алтернативи, критерий, избран вариант, цена на избора. Таблицата се попълва след като изборите бъдат потвърдени при реализацията, за да не описва

намерения вместо действия.]

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

Системата е проектирана около едно ограничение: описанието на услугата трябва да е едно, а различията между двамата доставчици трябва да са изнесени в ясно очертани места, чийто брой може да бъде преброен. Ако различията се разпръснат из цялото описание, сравнението губи смисъл, защото вече не се сравняват две среди, а две различни системи.

Архитектурата има четири слоя. *Изчислителният работник* е приложение на C++20, което приема заявка по HTTP, изпълнява изчислително интензивна задача с известна сложност и връща резултата, като чете входните данни и записва изхода в обектно хранилище. Той не знае при кой доставчик работи: адресът на хранилището и начинът за получаване на маркер за достъп идват от променливи на средата. *Контейнерният образ* е един и същ за двете разгръщания и се публикува в регистъра на съответния доставчик. *Описанието на инфраструктурата* се състои от обща коренна конфигурация и два модула, по един за всеки доставчик, с еднакъв интерфейс от входни променливи и изходи. *Слоят за наблюдение* събира метрики от работника и от клъстера.

Разделението между общото и платформено зависимото минава точно по границата на модулите за инфраструктура. Общи остават: описанието на работното натоварване за Kubernetes, правилото за хоризонтално мащабиране, сценарият за натоварване, конфигурацията на наблюдението и самият контейнерен образ. Платформено зависими са: създаването на мрежата, създаването на управлявания клъстер, създаването на кофата или контейнера за обекти и обвързването на идентичността, с която работникът получава достъп до хранилището без записан секрет.

3.1. Модел на идентичността и достъпа

Работникът никога не притежава дълготраен ключ. При всеки доставчик той получава краткосрочен маркер чрез механизма за федерация на идентичността между управляващия слой на клъстера и системата за управление на достъпа. Правото, което маркерът носи, е сведено до четене и запис в едно конкретно хранилище, без никакво друго действие. Този модел изравнява двете разгръщания по нещо съществено: и при двамата доставчици в изходния текст на проекта не съществува секрет.

[TODO: Да се посочат точните имена на механизмите при двамата доставчици,

както са в официалната документация към момента на реализацията, и да се приложи описание на издаваните права.]

3.2. Функционална схема

[TODO: Да се добави фигура със структурата на системата: генератор на натоварване, входна точка, група от реплики на работника, обектно хранилище, слой за наблюдение, и обозначени граници между общата и платформено зависимата част. Фигурата се означава по A.9 и към нея се прави препратка от текста.]

3.3. Договор на модулите за инфраструктура

Двата модула приемат едни и същи входни променливи: име на разгръщането, регион, начален и максимален брой реплики, размер на възела и етикет на контейнерния образ. Двата модула връщат едни и същи изходи: адрес на входната точка, име на хранилището за обекти и данни за връзка с клъстера. Заради този еднакъв договор преминаването от единия доставчик към другия е смяна на стойността на един параметър.

[TODO: Да се опише как се съпоставят типовете възли при двамата доставчици, тъй като пряко еднакви типове не съществуват. Съпоставянето влияе пряко на сравнението по цена и трябва да бъде записано преди измерванията, а не след тях.]

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

Исходният текст е разделен по отговорност, а не по вид на файловете. Директорията `src/` съдържа изчислителния работник, `include/` публичните му заглавни файлове, `tests/` модулните тестове, `infra/` описанието на инфраструктурата, а `load/` сценария за натоварване. Това разделение позволява всяка част да бъде изградена и проверявана самостоятелно.

Изчислителният работник е реализиран на C++20. Изборът на език следва от естеството на задачата: натоварването е изчислително интензивно, а не входно изходно, така че времето за реакция се определя от самото изчисление и всеки допълнителен слой между кода и процесора се отразява пряко на резултата. Съвсем практическо съображение е и това, че при C++ моментът на връщане на ресурсите е предвидим, което опростява тълкуването на измерванията: колебанията в закъснението не се дължат на паузи от събиране на боклука.

Работникът излага три крайни точки: една за приемане на заявка за изчисление, една за проверка на живост, използвана от оркестратора, и една, която публикува метрики в текстов формат, разпознаваем от системата за наблюдение. Метриките включват брой обработени заявки, разпределение на времето за обработка и брой едновременно обработвани заявки. Именно последната метрика управлява хоризонталното мащабиране, защото натоварването на процесора при изчислително интензивна задача остава близо до пълното и не носи информация.

[TODO: Да се опише конкретната изчислителна задача, която работникът изпълнява, заедно с нейната алгоритмична сложност и с параметъра, който определя обема на една единица работа. Единицата работа е основата на сравнението по цена и трябва да бъде определена еднозначно.]

4.1. Контейнеризация

Контейнерният образ се изгражда на два етапа: първият компилира приложението със статично свързване на зависимостите, а вторият копира само получения изпълним файл в минимална основа. Целта е малък образ, кратко време за изтегляне при мащабиране и малка повърхност за атака. Образът е един и същ за двата доставчика и се идентифицира чрез контролна сума, а не чрез етикет, за да не може двете разгръщания да се окажат с

различно съдържание.

[TODO: Да се приложи листинг на файла за изграждане на образа и да се посочи размерът на получения образ след измерване.]

4.2. Описание на инфраструктурата

Кореновата конфигурация избира модул според стойността на променлива за доставчик и подава към него еднакъв набор от входни променливи. Състоянието на инфраструктурата се съхранява отдалечено, в обектно хранилище при съответния доставчик, със заключване, за да не могат две едновременни изпълнения да го повредят.

[TODO: Да се приложат листинги на съществените части от двата модула и да се посочи броят редове на общата част спрямо платформено зависимата, като количествена мярка за това докъде стига преносимостта на описанието.]

4.3. Мерки срещу изтичане на данни за достъп

В хранилището не се записват идентификатори на сметки, абонаменти, имена на ресурси с глобален обхват или ключове. Всички такива стойности се подават чрез променливи на средата, а файлът `.env.example` съдържа само имената им със заместващи стойности. Файловете със състояние и с локални променливи са изключени от версионизирането.

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНО СРАВНЕНИЕ

Сравнението между двама доставчици има смисъл само ако разликата в измереното може да бъде отдадена на средата, а не на разликите в настройката. Затова методиката се записва преди провеждането на измерванията и всяко отклонение от нея се отбелязва явно в Раздел VI.

5.1. Условия за валидност на едно измерване

Едно измерване се приема за валидно, ако са изпълнени всички следни условия едновременно: двете разгръщания използват контейнерен образ с една и съща контролна сума; съпоставените типове възли са тези, определени в Раздел III; регионите са географски съпоставими; генераторът на натоварване е един и същ и се изпълнява от една и съща машина; правилото за мащабиране има еднакви прагове и еднакви граници; и измерването започва след установен период на загряване, през който резултатите се отхвърлят. Измерване, което не отговаря на някое от тези условия, не се сравнява, а се отбелязва като невалидно и се повтаря.

[TODO: Да се определят конкретните стойности: продължителност на загряването, продължителност на едно измерване, брой повторения, и правилото за отхвърляне на отклоняващи се стойности.]

5.2. Планирани експерименти

Предвидени са три експеримента. Първият измерва *закъснението при постоянно натоварване*: фиксиран брой заявки в секунда срещу фиксиран брой реплики, за да се получи базово сравнение без участие на мащабирането. Вторият измерва *поведението при мащабиране*: стъпаловидно нарастване на натоварването и отчитане на времето от преминаването на прага до момента, в който новите реплики започват да поемат трафик. Третият измерва *цената за единица извършена работа*: при еднакъв общ обем работа се отчита начислената от доставчика сума и се разделя на броя изпълнени единици.

[TODO: Да се уточни как се отчита начислената сума, тъй като таксуването при двамата доставчици се разпределя във времето по различен начин, и да се определи периодът, след който отчетът се приема за окончателен.]

5.3. Отчитани величини

За всеки експеримент се отчитат: закъснение по персентили, а не средна стойност, защото при опашки средната стойност крие точно това, което ни интересува; пропускателна способност в изпълнени единици работа за секунда; брой активни реплики във времето; и начислена сума за периода на измерването. Всяка величина се записва заедно с времето на измерването и с идентификатор на разгръщането, при което е получена.

5.4. Известни ограничения на методиката

Методиката не изолира мрежовото разстояние между генератора на натоварване и двата региона, което влияе на абсолютните стойности на закъснението. Тя не изолира и евентуалното различие в поколението на процесора между съпоставените типове възли, което е извън контрола на потребителя при двамата доставчици. И двете ограничения се посочват при тълкуването на резултатите, вместо да бъдат премълчани.

[TODO: Да се прецени дали мрежовото разстояние да бъде извадено от отчетеното закъснение чрез отделно измерване на времето за обръщение до всяка входна точка, и ако да, да се опише как.]

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

Разделът представя получените резултати в реда на експериментите от Раздел V. Всяка таблица се придружава от коментар в текста и от препратка към нея. Стойностите се попълват след провеждане на измерванията и не се предвиждат предварително.

6.1. Закъснение при постоянно натоварване

[TODO: Таблица със структура: доставчик, брой заявки в секунда, закъснение на 50-и, 90-и, 95-и и 99-и персентил, дял на неуспешните заявки. По един ред за всеки доставчик и всяко ниво на натоварването. Стойностите се попълват след измерването.]

[TODO: Фигура: разпределение на закъснението за двата доставчика, наложени на една координатна система, с легенда по A.9.]

[TODO: Коментар: коя от двете среди дава по-ниско закъснение, на кой персентил разликата е най-голяма, и попада ли разликата в границите на разсейването между повторенията.]

6.2. Поведение при хоризонтално мащабиране

[TODO: Таблица със структура: доставчик, време от преминаването на прага до създаване на нова реплика, време до готовност на репликата, време до поемане на трафик, общо време за реакция. По един ред за всяко повторение.]

[TODO: Фигура: брой активни реплики и закъснение във времето при стъпаловидно нарастване на натоварването, за двата доставчика.]

[TODO: Коментар: къде отива по-голямата част от времето за реакция, и се ли дължи разликата между доставчиците на управляващия слой или на времето за подготовка на възела.]

6.3. Цена за единица извършена работа

[TODO: Таблица със структура: доставчик, компонент на разхода, начислена сума за периода, брой изпълнени единици работа, цена за единица. Компонентите се изброяват поотделно, тъй като разпределението между изчисление, съхранение и мрежов трафик е част от резултата, а не подробност.]

[TODO: Коментар: коя среда излиза по-евтина за същата работа, кой компонент отговаря за разликата, и променя ли се изводът при по-голям обем.]

6.4. Обобщен анализ

[TODO: Обобщение на трите експеримента и отговор на въпроса, поставен в увода: докъде едно декларативно описание остава едно, и колко от разликата между двамата доставчици е видима за потребителя на услугата. Тук се посочват и всички отклонения от методиката, ако е имало такива.]

ЗАКЛЮЧЕНИЕ

Проектът изгражда изчислително интензивна услуга, разгърната едновременно при двама публични облачни доставчика от едно декларативно описание на инфраструктурата, и измерва разликата между двете среди при контролирано еднакви условия. Разделението между общата и платформено зависимата част е направено така, че различията между доставчиците да са локализирани в два модула с еднакъв интерфейс, а всичко останало, включително контейнерният образ, описанието на работното натоварване и правилото за мащабиране, да остане едно.

Предимства на избраното решение. Смяната на доставчика е смяна на стойността на един параметър, което прави сравнението осмислено и повторяемо. Метриците се събират от самото приложение, с един и същ код при двете разгръщания, така че разликата в измереното не се дължи на разлика в измервателния уред. В хранилището с изходния текст не се съхранява нито един секрет.

Недостатъци и ограничения. Изборът на управляван Kubernetes дава най-голяма преносимост на описанието, но и най-висока постоянна такса, така че изводът за цената важи за този начин на изпълнение, а не за облака изобщо. Съпоставянето на типове възли между двамата доставчици е приблизително, защото еднакви типове не съществуват. Мрежовото разстояние до двата региона не е изолирано напълно.

[TODO: Да се обобщят получените резултати с оглед на целта, поставена в увода, след като измерванията бъдат проведени. Тук се посочва и кои от предварителните очаквания не са се потвърдили, ако има такива.]

Насоки за по-нататъшна работа. Естественото продължение е разширяване на сравнението с трети доставчик, добавяне на второ, входно изходно интензивно натоварване, при което гаранциите на обектното хранилище влизат в сила по различен начин, както и проверка на поведението при отказ на цяла зона на достъпност.

ЛІТЕРАТУРА

- [1] P. Mell and T. Grance, “The NIST definition of cloud computing,” Tech. Rep. Special Publication 800-145, National Institute of Standards and Technology, Gaithersburg, MD, USA, September 2011.
- [2] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, and M. Zaharia, “A view of cloud computing,” *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [3] P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt, and A. Warfield, “Xen and the art of virtualization,” in *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP '03)*, pp. 164–177, 2003.
- [4] D. Merkel, “Docker: Lightweight Linux containers for consistent development and deployment,” *Linux Journal*, vol. 2014, no. 239, 2014.
- [5] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, “Firecracker: Lightweight virtualization for serverless applications,” in *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20)*, pp. 419–434, 2020.
- [6] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, “Large-scale cluster management at Google with Borg,” in *Proceedings of the 10th European Conference on Computer Systems (EuroSys '15)*, 2015.
- [7] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, omega, and kubernetes,” *ACM Queue*, vol. 14, no. 1, 2016.
- [8] I. Foster, C. Kesselman, and S. Tuecke, “The anatomy of the grid: Enabling scalable virtual organizations,” *International Journal of High Performance Computing Applications*, vol. 15, no. 3, pp. 200–222, 2001.
- [9] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels, “Dynamo: Amazon’s highly available key-value store,” in *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP '07)*, pp. 205–220, 2007.

- [10] B. Calder, J. Wang, A. Ogus, N. Nilakantan, A. Skjolsvold, S. McKelvie, Y. Xu, S. Srivastav, J. Wu, H. Simitci, *et al.*, “Windows Azure storage: A highly available cloud storage service with strong consistency,” in *Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP '11)*, pp. 143–157, 2011.
- [11] HashiCorp, “Terraform documentation.” <https://developer.hashicorp.com/terraform/docs>.
посетен на [TODO: дата на достъп].
- [12] The Kubernetes Authors, “Kubernetes documentation.” <https://kubernetes.io/docs/>.
посетен на [TODO: дата на достъп].

ПРИЛОЖЕНИЯ

Приложение А. Изходен текст на изчислителния работник

[TODO: Да се приложи изходният текст на работника, или съществените му части, чрез `\lstinputlisting` от `src/`. Пълният текст е в хранилището на проекта.]

Приложение Б. Описание на инфраструктурата

[TODO: Да се приложат кореновата конфигурация и двата модула за доставчици от `infra/`, без стойности на променливи, съдържащи идентификатори.]

Приложение В. Сценарий за синтетично натоварване

[TODO: Да се приложи сценарият от `load/`, заедно с точните параметри, с които е изпълнен при всяко измерване.]

Приложение Г. Сурови резултати от измерванията

[TODO: Да се приложат необработените изходни данни от измерванията, за да може получената в Раздел VI обработка да бъде проверена независимо.]

Приложение Д. Екранни форми

[TODO: Да се приложат екранни изображения от таблото за наблюдение при всеки от трите експеримента, с обозначен доставчик и време на заснемане.]